

This work is licensed under a Creative Commons license



Attribution-NonCommercial-NoDerivatives 4.0 International
(CC BY-NC-ND 4.0)

You are free to:

- **Share** – copy and redistribute the material in any medium or format.

Under the following terms:

- **Attribution** – You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.
- **NonCommercial** – You may not use the material for commercial purposes.
- **NoDerivatives** – If you remix, transform, or build upon the material, you may not distribute the modified material.

Instruction Set Architecture (ISA)

Part III – Intel x86/x64

Simone Aonzo, PhD



Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

Instruction Set Architecture (ISA)

The ISA is the design of how machine code works

- An abstract model of a computer
 - $\Rightarrow$ interface between software and hardware
- The realization of an ISA is called an *implementation*
- New processors are designed to be backward compatible
- A software written for a specific ISA can run on different implementations; for example:
 - Intel and AMD implement processors of the x86 ISA
 - Apple, Qualcomm, and Samsung implement processors of the ARM ISA
 - E.g., Apple M*, Snapdragon, Exynos

History of ISA varieties

- **x86 (CISC)**
 - 1974: Intel 8080, 8-bit
 - 1978: Intel 8086, 16-bit
 - 1985: Intel 80386, 32-bit (aka i386, IA-32)
 - 2003: AMD Opteron, 64-bit (aka x64, x86_64, AMD64, Intel 64)
- **ARM (RISC)**
 - 1985: ARMv1, 32-bit
 - 2011: ARMv8-A, 64-bit
 - 2013: the iPhone 5S is the first 64-bit smartphone
 - 2020: the Apple M1 is the first ARM processor for the Mac
- **MIPS (RISC) - introduced in 1985**
 - Still used in embedded systems such as routers
 - Old consoles: Nintendo 64, PSX, PS2, PSP
- **PowerPC (RISC) - introduced in 1992, now named PowerISA**
 - Apple's Macintosh computers until 2006, then switched to the x86
 - Old consoles: Wii, PS3, and Xbox 360
- **RISC-V (RISC) - introduced in 2010**
 - Royalty-free and open-source ISA that can be extended or customized

The rise of RISC

In the old days (around the 1970s)

- Memories were small and extremely expensive
- Compilers were bad
- $\implies$ powerful machine instructions help to address both issues

Then, RAM got cheaper and compilers got better

- Idea: all-in with compiler's optimizations
- Combine simple instructions rather than choosing the best from a complex set
- $\implies$ simpler electronic implementation

Technology continues to evolve...

- x86 and ARM have stolen ideas from each other, blurring differences
- LoongArch is a new RISC ISA (inspired by MIPS and RISC-V)
 - Developed to reduce Chinese reliance on western technology
 - E.g. Loongson processors are by default used in government systems

- Privilege Levels & Modes of Operation
- Instruction Types
- Registers
- Memory Model
- Execution Model
- Calling Conventions & Stack Frames

x86/ARM common concepts – Privilege Levels & Modes

Each CPU mode has distinct rights for accessing memory, I/O and instructions.
In descending order of privileges:

- **x86 rings:**

- Ring -2 – System Management Mode
- Ring -1 – Hypervisor (Intel VT-x / AMD-V VMX)
- Ring 0 – OS Kernel (**kernel** mode)
- Rings 1 and 2 – exist in theory, but unused by modern OSes
- Ring 3 – Applications (**user** mode)

- **ARM Exception Levels (EL):**

- EL3 – Secure Monitor
- EL2 – Hypervisor
- EL1 – OS Kernel (**kernel** mode)
- EL0 – Applications (**user** mode)

- **Data Movement:** PUSH, POP, MOV/MOVB, MOV/LDR|STR
- **Arithmetic and Logic:** ADD, SUB, AND, OR, XOR, ...
- **Control Flow**
 - Unconditional jumps (e.g., JMP/B)
 - Calls (CALL/BL)
 - Conditional branches (e.g., JZ/BEQ, JNZ/BNE)
- **SIMD (Single Instruction, Multiple Data):** data-level parallelism
 - i.e., n operations at once
 - image/audio/video processing, cryptography, vector math, ...

- **General-purpose registers** for integer and pointer operations
 - x86
 - **32bit:** 8 32-bit-wide registers (EAX, EBX, ..., EBP)
 - **64bit:** 16 64-bit-wide registers (RAX, RBX, ..., RBP, R8, ..., R15)
 - ARM
 - **32bit:** 16 32-bit-wide registers (R0–R15)
 - **64bit:** 31 64-bit-wide registers (X0–X30)
- **Special-purpose registers:**
 - Instruction Pointer / Program Counter → points to (address of) the current instruction
 - Stack Pointer → points to (address of) the current top of the stack
 - Flags / Status register → holds condition flags for branching

4-stage pipeline

- ❶ **Fetch:** reads the next instruction from memory
 - Into the Instruction Pointer / Program Counter
 - *Branch prediction* guesses the next instruction when encountering jumps/branches
- ❷ **Decode:** raw instruction bits are interpreted
 - Broken into fields (opcode, registers, immediates)
 - Converted into internal “micro-ops” or control signal
 - *Macro-fusion (x86)* combines certain instruction pairs into a single micro-op
- ❸ **Execute:** execution units perform the computation
 - e.g., ALUs, FPUs, SIMD
 - **Out-of-order execution** runs ready instructions while others may wait
 - **Speculative execution** executes predicted branch, rolling back if necessary
- ❹ **Write-back:** results are committed back into registers/memory

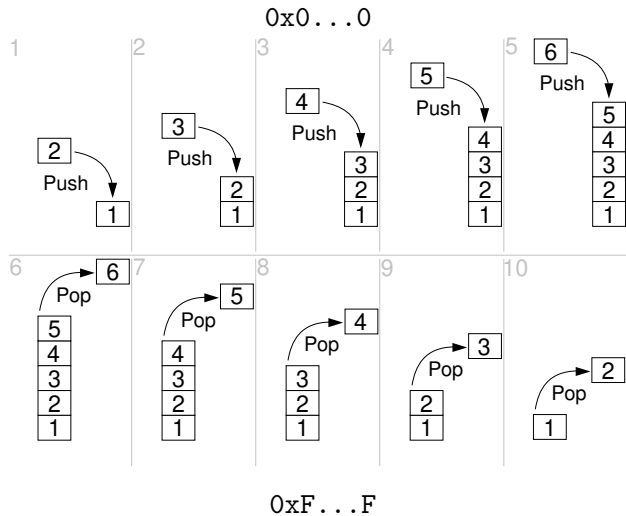
Calling Convention: define how a program pass arguments, return values, and preserve registers when calls a function

- ARM: sticks to ARM Procedure Call Standard
- x86: plethora of conventions 🏰
 - cdecl, stdcall, fastcall, vectorcall, Microsoft's thiscall, Windows ABI, ...

The **Stack** is *downward-growing* memory region for managing function calls

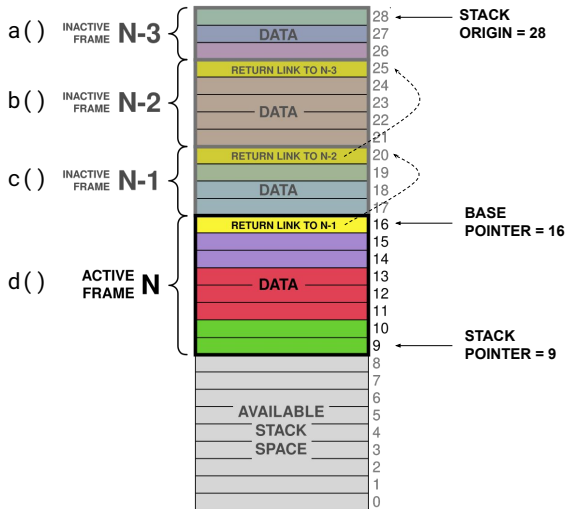
- Each thread (inside a process) has its own
- Every time a function is called the code creates a **stack frame**
 - It is just a convention $\implies$ may not be respected
- Manipulated with PUSH/POP instructions

PUSH/POP on the Stack (downward-growing)



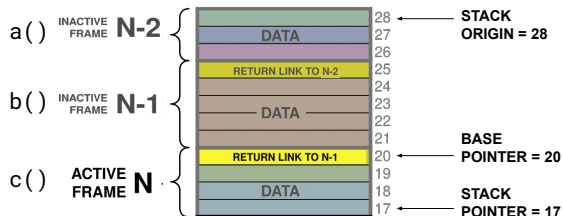
Stack Frames [1/2]

E.g., some code calls 4 functions in sequence: `a()`; `b()`; `c()`; `d()`;



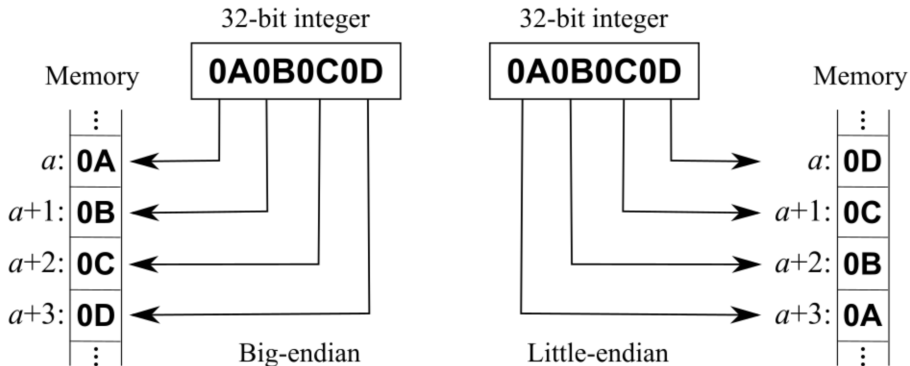
Stack Frames [2/2]

E.g., function `d()` ends, the frame of `c()` is restored



x86/ARM common concepts – Endianness

How data is stored in (main) memory?



- x86 → always little-endian
- ARM → support both, but today only little-endian is used

- **Byte-addressable** unified memory architecture
 - E.g., storing a 32-bit integer 0x0A0B0C0D in little-endian
 - 0x1000 → 0D
 - 0x1001 → 0C
 - 0x1002 → 0B
 - 0x1003 → 0A
- **Memory Management Unit (MMU)**
 - MMU translates *virtual addresses* (used by programs) into *physical addresses* (actual RAM)
 - Page tables define these mappings in fixed-size *pages* (typically 4 KiB)
 - Each page table entry carries attribute bits, e.g.
 - R/W: Read-only vs. read/write permissions
 - U/S: User vs. supervisor (kernel) mode access
 - eXecute Disable/Never (XD/XN) disable page execution
- Cache hierarchies (L1/L2/L3) with coherent protocols across cores

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

A little history of Intel CPUs

- 1978 16-bit processors: 8086 and 8088 (8-bit bus); segmentation, $2^{20} = 1$ MB address space. Since then **words** = 16 bit ☺
- 1982 286, protected mode (segment selectors + descriptor tables), $2^{24} = 16$ MB space
- 1985 386, 32-bit processor, virtual-8086 mode, $2^{32} = 4$ GB address space, segmented-memory model and flat memory model, paging with 4 KB pages
- 1989 486, integrated x87 FPU
- 1993 Pentium, 4k and 4M pages
- 1995-1999 P6 family, MMX and SSE → SIMD parallelism
- 2000-2007 Pentium4/Xeon family, SSE2 & SSE3; AMD64/Intel 64, hyperthreading and VT
- 2008 Core i7 family, SSE4.2, 2nd generation Virtualization Technology
- ...

Modes of operation 16/32/64

- 16 16-bit real-address mode, the “8086 mode”; activated at power-up. If interested, watch: [35C3 - A deep dive into the world of DOS viruses](#)
- 32 protected mode, the “normal” mode
 - four protection rings, two used: 0 (kernel) and 3 (user)
 - applications run with a paged 32-bit flat address space
- 64 IA32e, with two submodes:
 - compatibility mode, similar to 32-bit protected mode, permits legacy 16/32-bit application to run without recompilation on a 64-bit OS
 - 64-bit mode, allows to run 64-bit applications

CPU: the Intel x86/x64

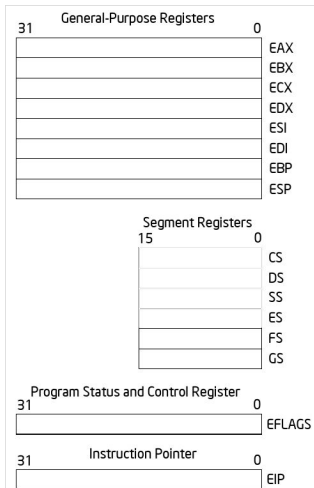
We mostly deal with **user-mode** of:

- **x86**/IA-32 (“Intel Architecture 32”, also called i386)
- **x86-64/x64**/Intel 64/AMD64 is an extension to original IA-32
 - Beware: IA-64 (Intel Architecture 64)/Itanium is an almost completely unrelated 64 bit architecture

Documented in Intel **Software Developer Manuals**

- at the time of writing, a handy 5000+ (!) page reference
- hundreds of opcodes, *however*
- **5** of them cover about 64% of opcodes in “normal programs” [B⁺06]
 - namely: mov, push, pop, call, cmp
 - **14** opcodes cover about 90%, and 72 cover > 99.8%

(Basic) Registers in 32 bit mode



(Mostly) General purpose:

- EAX – **A**ccumulator
- EBX – pointer to data
- ECX – **C**ounter
- EDX – calling convention / arithmetic
- ESI – **S**ource pointer for string ops
- EDI – **D**estination pointer

Execution registers:

- EBP – **B**ase/frame **P**ointer
- ESP – **S**tack **P**ointer
- EIP – **I**nstruction **P**ointer
- EFLAGS – carry, sign, zero, parity, ...

General Purpose Registers in 32 bit mode

General-Purpose Registers

31	16	15	8	7	0	16-bit	32-bit
	AH		AL			AX	EAX
	BH		BL			BX	EBX
	CH		CL			CX	ECX
	DH		DL			DX	EDX
	BP						EBP
	SI						ESI
	DI						EDI
	SP						ESP

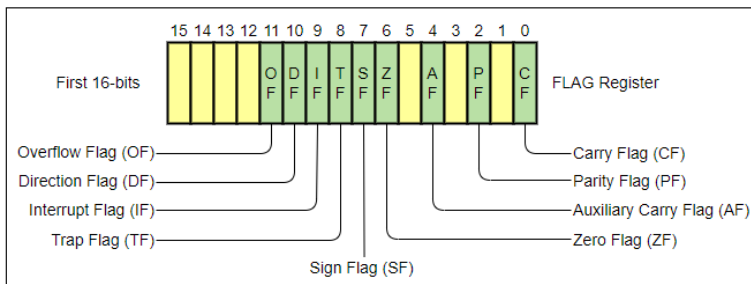
General Purpose Registers in 64 bit mode

- sixteen 64-bit registers **R0–R15** [D,W,B]
 - R0–R7 are aliases to old ones
 - R0=**RAX**, R0D=**EAX**, R0W=**AX**, R0B=**AL** (no R0... for AH)
- Assigning the 32 lower bits, zero-extends the upper ones
- Assigning the 16 or 8 lower bits, preserves the upper ones

6 3		3 2	3 1		1 6	1 5		8 7		0
RAX or R0										
zero-extended				EAX or R0D						
preserved				preserved			AX or R0W			
							AH		AL or R0B	
RCX or R1										
zero-extended				ECX or R1D						
preserved				preserved			CX or R1W			
							CH		CL or R1B	
RDX or R2										
zero-extended				EDX or R2D						
preserved				preserved			DX or R2W			
							DH		DL or R2B	

(R|E)FLAGS Register

$|\text{FLAGS}| = 16 \text{ bit}$, $|\text{EFLAGS}| = 32 \text{ bit}$, $|\text{RFLAGS}| = 64 \text{ bit}$



Bit vector. The most relevant are set when:

- CF – Carry Flag: unsigned carry/borrow
- ZF – Zero Flag: the result of an operation is zero
- SF – Sign Flag: the sign of the result (most significant bit)
- OF – Overflow Flag: signed overflow

Segment Registers

Technically, six 16-bit segment registers: CS, DS, ES, SS, FS and GS.

Enabled >64KB memory access (real mode) and memory protection.
Today largely disabled in 64-bit mode; only FS/GS base used.

However, modern OSes use **paging** (only) → **virtual flat address space**

- FS and GS used for TLS|TIB

Linux FS in 64-bit mode, GS in 32-bit mode

https://wiki.osdev.org/Thread_Local_Storage

Windows viceversa, FS in 32-bit mode, GS in 64-bit mode

https://en.wikipedia.org/wiki/Win32_Thread_Information_Block

Operations and Operands

Operations (machine-instructions) act on zero or more operands

- Some are specified explicitly and others are implicit
- *Usually* x86 instructions are in this form:
- **operation** **operand**_{destination} **operand**_{source}

The data for an operand can be located in:

- the instruction itself, in case of an **immediate** operand
 - immediate → constant value encoded directly in the instruction
- a **register**
- a **memory location**

when there are more operands, **only one** can refer to memory

MOVe instruction

Copies the second operand (source) to the first operand (destination)

- `mov eax, 1`
 - copies the immediate 1 into register eax
 - $\text{eax} \leftarrow 1$
- `mov bl, al`
 - copies the value in al into bl
 - $\text{ebx} \leftarrow \text{eax}$
- `mov eax, [ecx]`
 - square brackets [...] indicate memory access (dereferencing)
 - copies the value at address ecx into eax
 - $\text{eax} \leftarrow * \text{ecx}$

MOV is the most common x86 instruction: about 20% of instructions in a program

Surprisingly, programs using only `mov` can be Turing-complete!!! [Dol13]

Addressing modes

Addressing modes for 16-bit x86 processors can be summarized by the formula:^{[15][16]}

$$\begin{Bmatrix} \text{CS :} \\ \text{DS :} \\ \text{SS :} \\ \text{ES :} \end{Bmatrix} \begin{Bmatrix} \{ \text{BX} \} \\ \{ \text{BP} \} \end{Bmatrix} + \begin{Bmatrix} \{ \text{SI} \} \\ \{ \text{DI} \} \end{Bmatrix} + [\text{displacement}]$$

Addressing modes for 32-bit x86 processors,^[17] and for 32-bit code on 64-bit x86 processors, can be summarized by the formula:^[18]

$$\begin{Bmatrix} \text{CS :} \\ \text{DS :} \\ \text{SS :} \\ \text{ES :} \\ \text{FS :} \\ \text{GS :} \end{Bmatrix} \begin{Bmatrix} \begin{Bmatrix} \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{ESP} \\ \text{EBP} \\ \text{ESI} \\ \text{EDI} \end{Bmatrix} \end{Bmatrix} + \begin{Bmatrix} \begin{Bmatrix} \text{EAX} \\ \text{EBX} \\ \text{ECX} \\ \text{EDX} \\ \text{EBP} \\ \text{ESI} \\ \text{EDI} \end{Bmatrix} * \begin{Bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{Bmatrix} \end{Bmatrix} + [\text{displacement}]$$

Addressing modes for 64-bit code on 64-bit x86 processors can be summarized by the formula:^[18]

$$\begin{Bmatrix} \begin{Bmatrix} \text{FS :} \\ \text{GS :} \end{Bmatrix} [\text{general register}] + \begin{Bmatrix} \text{general register} * \begin{Bmatrix} 1 \\ 2 \\ 4 \\ 8 \end{Bmatrix} \end{Bmatrix} \end{Bmatrix} + [\text{displacement}]$$

RIP

https://en.wikipedia.org/wiki/X86#Addressing_modes

Quiz 0: mov into register

Initial value, memory at 0x1018: 0xDEADBEEFCAFEBAFE

```
mov    rax, 0xFFFFFFFFFFFFFFFF
mov    al, 0x11
mov    ah, 0x22
mov    ax, 0x3344
mov    eax, 0x55667788
```

How is the content of rax evolving?

Quiz 0: solution

```
mov    rax, 0xFFFFFFFFFFFFFFFF ; rax = 0xffffffffffffffff
mov    al, 0x11                 ; rax = 0xffffffffffff11
mov    ah, 0x22                 ; rax = 0xffffffffffff2211
mov    ax, 0x3344               ; rax = 0xffffffffffff3344
mov    eax, 0x55667788          ; rax = 0x55667788
```

Reasoning

In x86-64, writing to any 32-bit general-purpose register (e.g., `eax`, `ebx`, `r8d`) automatically zeroes the upper 32 bits of the corresponding 64-bit register, while writes to 8- or 16-bit subregisters do not affect the upper bits

LEA – Load Effective Address

LEA *r*, *m*

- Computes the memory address of the source operand
- Stores that numeric value into the destination register
- Does **not** access memory (unlike MOV)

Typical uses:

- Get the address of a variable/label (e.g., pointer to a static string)
`lea eax, [var] ; eax = &var`
- Trick: arithmetic using addressing modes w/out updating the FLAGS
`lea eax, [ebx+ecx*4] ; eax = ebx+ecx*4`
- In 64-bit mode (in 32 is invalid reading RIP), obtain current RIP:
`lea rax, [rip+0] ; rax = address of next instruction`

IP-relative addressing

The instruction encodes a displacement that is added to the current RIP to form the effective address $\Rightarrow$ code/data references to be expressed relative to where the code is loaded

Quiz 1: mov and lea

Initial value, memory at 0x1018: 0xDEADBEEFCAFEBAFE

```
mov ecx, 4  
mov rax, 0x1000  
lea rax, [rax + rcx*4 + 8]
```

What is the final value of rax?

- ① 0x1018
- ② 0xDEADBEEFCAFEBAFE

Quiz 1: solution

```
mov ecx, 4           ; rcx = 4
mov rax, 0x1000       ; rax = 0x1000
lea rax, [rax + rcx*4 + 8] ; rax = 0x1000 + 4 * 0x4 + 0x8 = 0x1018
```

Reasoning

lea computes the effective address:

$$\text{rax} = 0\text{x}1000 + 4 \cdot 0\text{x}4 + 0\text{x}8 = 0\text{x}1018$$

It **does not read memory** at 0x1018

Correct answer: **1. 0x1018**

Two syntaxes for assembly: Intel vs AT&T

- Source-destination ordering:
 - Intel: destination $\leftarrow$ source
 - AT&T: source $\leftarrow$ destination
- Register naming: AT&T prefixes register names with %
- Immediate operands: AT&T prefixes immediate operands with \$
- Operand size
 - in AT&T determined from the last character of the op-code name
 - Intel prefixes memory operands with *size ptr*
 - optional when there are no ambiguities
- Memory operands
 - in Intel syntax the base register is enclosed in [and]
 - in AT&T they change to (and)
- Indirect Memory References
 - Intel: [base + index*scale + disp]
 - AT&T: disp(base, index, scale)

Intel vs AT&T: examples

Intel Code			AT&T Code	
-----		+	-----	
mov	eax,1		movl	\$1,%eax
mov	ebx,0ffh		movl	\$0xff,%ebx
int	80h		int	\$0x80
mov	ebx, eax		movl	%eax, %ebx
mov	eax,[ecx]		movl	(%ecx),%eax
mov	eax,[ebx+3]		movl	3(%ebx),%eax
mov	eax,[ebx+20h]		movl	0x20(%ebx),%eax
add	eax,[ebx+ecx*2h]		addl	(%ebx,%ecx,0x2),%eax
lea	eax,[ebx+ecx]		leal	(%ebx,%ecx),%eax
sub	eax,[ebx+ecx*4h-20h]		subl	-0x20(%ebx,%ecx,0x4),%eax

Intel syntax is the de facto standard in SRE (for its readability)

<https://www.ibiblio.org/gferg/ldp/GCC-Inline-Assembly-HOWTO.html#s3>

Size matters :)

- Registers have inherent size: e.g., (R|E)AX
 - `mov eax, edi` $\Rightarrow$ both 32-bit by the register names
- Memory addresses do not: e.g., `mov [rbp-4], edi`
 - $\Rightarrow$ *how many bytes are we MOVing in [rbp-4]?*
- (Dis)Assemblers require an **explicit size** for memory operands
 - BYTE = 1 Byte = 8 bits
 - WORD = 2 Byte = 16 bits
 - DWORD = 4 Byte = 32 bits
 - QWORD = 8 Byte = 64 bits
- In Intel-style syntax, always followed by PTR (PoinTeR). E.g.,
 - `mov DWORD PTR [rbp-4], edi` (32-bit store)
 - `mov QWORD PTR [rbp-8], rax` (64-bit store)
- If operand size is unambiguous or inferable disassemblers may omit it for readability

How to read the documentation

*“Give a man a fish and you feed him for a day;
teach a man to fish and you feed him for a lifetime”*

- Official Intel Software Developer Manuals (SDM)
 - Volume 2: Includes the full instruction set reference
 - About 5k pages 😊
- x86 and amd64 instruction reference by Félix Cloutier [[link](#)]
 - Opcode, Instruction, Op/En (Operand/Encoding)
 - I = Immediate, R = Registry, M = Memory
 - E.g., `ADD r/m32, imm32`
 - Performs `r/m32 = r/m32 + imm32`
 - `r/m32` can be a 32-bit register or a memory location
- AI chatbot
 - Explain, examples, ...

x86 Integer Arithmetic – Addition / Subtraction

- **ADD** *r/m*, *r/imm* → **ADD**
 - $dest = dest + src$
- **ADC** *r/m*, *r/imm* → **ADd** with **Carry** (CF flag)
 - $dest = dest + src + CF$
- **SUB** *r/m*, *r/imm* → **SUBb**tract
 - $dest = dest - src$
- **CMP** *r/m*, *r/imm* → **CoMP**are by subtracting but sets (R|E)FLAGS only
 - $dest - src$
- **SBB** *r/m*, *r/imm* → **SuB**tract with **B**orrow (CF flag)
 - $dest = dest - src - CF$
- **INC** *r/m* → **INC**rement
 - $dest = dest + 1$
- **DEC** *r/m* → **DEC**rement
 - $dest = dest - 1$
- **NEG** *r/m* → **NEG**ate $\implies$ arithmetic inverse (two's complement)
 - $dest = 0 - dest$ (CF=1 if operand $\neq 0$)

Quiz 2: straight-line arithmetic

```
mov     eax, 0x10
mov     ecx, 0x3
add     eax, ecx
inc     al
sub     rax, 5
dec     rax
```

What is the final value of rax?

- ① 0x0B
- ② 0x0E
- ③ 0x0F
- ④ 0x10

Quiz 2: solution

Step

```
mov    eax, 0x10    ; rax = 0x10
mov    ecx, 0x3     ; rcx = 0x3
add    eax, ecx     ; rax = rax + rcx = 0x10 + 0x3 = 0x13
inc    al           ; rax = rax + 1 = 0x13 + 1 = 0x14
sub    rax, 5       ; rax = rax - 5 = 0xf
dec    rax          ; rax = rax - 1 = 0xf - 1 = 0xe
```

Correct answer: **2. 0x0E**

Carry vs Overflow Flags: Examples

Reminder: AL is the low 8-bit subregister of RAX(64-bit) register

Instructions	AL=	CF	OF	Why?
<code>mov al,255; add al,1</code>	0	1	0	Unsigned carry
<code>mov al,1; sub al,2</code>	255	1	0	Unsigned borrow
<code>mov al,127; add al,1</code>	-128	0	1	Signed overflow
<code>mov al,-128; sub al,1</code>	127	0	1	Signed overflow

Arithmetic on large integers

ADC and **SBB** propagate the carry/borrow, allowing the CPU to perform addition/subtraction on integers larger than the register size. E.g., 128-bit using 64-bit registers

Quiz 3: unsigned carry

```
mov     eax, 0x1
neg     al
mov     ebx, 0x5
add     al, 1
adc     bl, 2
```

What is the final value of rbx?

- ① 0x07
- ② 0x08
- ③ 0x09
- ④ 0x00

Quiz 3: solution

```
mov    eax, 0x1    ; eax = 1
neg     al          ; al = 0 - 1 = 0xff (unsigned) = -1 (signed)
mov     ebx, 0x5    ; ebx = 5
add     al, 1       ; al = 0xff + 1 = 0x100 -> al=0x00 and CF=1
adc     bl, 2       ; bl = bl + 2 + CF = 5 + 2 + 1 = 8
```

Correct answer: **2. 0x08**

x86 Integer Arithmetic – Multiply / Divide

One operand is implicit, result stored in two registers

- **MUL** *r/m* — unsigned **MUL**tiply
 - (R|E)AX is an implicit operand
 - Result is stored in (R|E)DX:(R|E)AX (high:low)
 - $(R|E)AX * r/m \rightarrow (R|E)DX:(R|E)AX$
- **DIV** *r/m* — unsigned **DIV**ide
 - Implicit dividend in (R|E)DX:(R|E)AX
 - $(R|E)DX:(R|E)AX \div r/m \rightarrow$
 - Quotient (R|E)AX
 - Remainder (R|E)DX

x86 Bitwise Logic, Shifts and Rotates

Bitwise Logic

- **AND** *r/m, r/imm* — bitwise AND
- **TEST** *r/m, r/imm* $\approx$ AND — but sets (R|E)FLAGS only
 - **TEST** *x*, $x == 0 \iff x == 0$
- **OR** *r/m, r/imm* — bitwise OR
- **XOR** *r/m, r/imm* — bitwise XOR
- **NOT** *r/m* — bitwise inverse (one's complement)
 - $dest = 0 - dest - 1$

Shifts / Rotates

- **SHL|SAL** *r/m, imm/CL* — **S**Hift **L**eft, multiplies by 2^n
- **SHR** *r/m, imm/CL* — **S**Hift **R**ight, divides unsigned int by 2^n
- **SAR** *r/m, imm/CL* — **S**Hift **A**rithmetic **R**ight, divides signed int by 2^n preserving sign bit
- **ROL** *r/m, imm/CL* — **R**Otate **L**eft
- **ROR** *r/m, imm/CL* — **R**Otate **R**ight

Quiz 4: Bitlogic & Multiplication

```
xor    rax, rax
xor    rbx, rbx
mov    al, 6
mov    bl, 3
xor    rax, rbx
shl    rax, 2
mul    rbx
```

Question:

- What is the final value of `rax` (in decimal)?
- What is the value after each instruction?

Quiz 4: solution

```
xor    rax, rax    ; rax = 0
xor    rbx, rbx    ; rbx = 0
mov     al, 6       ; rax = 6
mov     bl, 3       ; rbx = 3
xor     rax, rbx    ; rax = rax XOR rbx = 6 XOR 3 = 5
shl     rax, 2       ; rax = rax << 2 = 5 * 2^2 = 20 = 0x14
mul     rbx          ; rax = rax * rbx = 20 * 3 = 60 = 0x3c
```

Correct answer: **0x3c**

Unconditional Jumps

- **JMP rel8/rel32** — short/near relative jump
 - E.g., `jmp +5`; skip next 5 bytes (short jump)
- **JMP r/m** — indirect jump (reg or memory)
 - E.g., `jmp rax` ; jump to address in rax
 - E.g., `jmp [rbx]` ; reads the value in memory at [rbx] and jump to it

Indirect jumps complicate program analysis

The jump target is determined at runtime (from a register or memory), making control-flow harder to predict and statically reconstruct

x86 Control Flow: Conditional Jumps (Jcc) based on (R|E)FLAGS

Jcc rel8/rel32 — Jump if:

Category	Aliases	Condition	Flags
Equality	JE $\equiv$ JZ	Equal / Zero	$ZF = 1$
Equality	JNE $\equiv$ JNZ	Not Equal / Not Zero	$ZF = 0$
Signed	JL $\equiv$ JNGE	Less	$SF \neq OF$
Signed	JLE $\equiv$ JNG	Less or Equal	$ZF = 1 \vee SF \neq OF$
Signed	JG $\equiv$ JNLE	Greater	$ZF = 0 \wedge SF = OF$
Signed	JGE $\equiv$ JNL	Greater or Equal	$SF = OF$
Unsigned	JB $\equiv$ JC $\equiv$ JNAE	Below	$CF = 1$
Unsigned	JBE $\equiv$ JNA	Below or Equal	$CF = 1 \vee ZF = 1$
Unsigned	JA $\equiv$ JNBE	Above	$CF = 0 \wedge ZF = 0$
Unsigned	JAE $\equiv$ JNC $\equiv$ JNB	Above or Equal	$CF = 0$

x86 Control Flow: Typical TEST/CMP Jcc patterns

- **TEST** $x, x \rightarrow$ computes $x \wedge x$, so it tests if x is zero
 - **JE** $\equiv$ **JZ** 0x1234 ; jump to 0x1234 if $x == 0$
 - **JNE** $\equiv$ **JNZ** 0x1234 ; jump to 0x1234 if $x \neq 0$
- **CMP** $x, y \rightarrow$ computes $x - y$ and sets FLAGS
 - $x < y$
 - **JL** 0x1234 ; jump to 0x1234 if $x < y$ (signed)
 - **JB** 0x1234 ; jump to 0x1234 if $x < y$ (unsigned)
 - $x \leq y$
 - **JLE** 0x1234 ; jump to 0x1234 if $x \leq y$ (signed)
 - **JBE** 0x1234 ; jump to 0x1234 if $x \leq y$ (unsigned)
 - $x > y$
 - **JG** 0x1234 ; jump to 0x1234 if $x > y$ (signed)
 - **JA** 0x1234 ; jump to 0x1234 if $x > y$ (unsigned)
 - $x \geq y$
 - **JGE** 0x1234 ; jump to 0x1234 if $x \geq y$ (signed)
 - **JAE** 0x1234 ; jump to 0x1234 if $x \geq y$ (unsigned)

Quiz 5: Control Flow

Question: How does `rax` change as `x` vary?

```
    mov rdi, x

    cmp    rdi, -3
    jl     .label1
    cmp    rdi, 2
    jg     .label1
    xor     rax, rax
    jmp    .label2
.label1:
    mov     eax, 1
.label2:
    ...
```

Quiz 5: solution

```
mov rdi, x

cmp    rdi, -3
jl     .label1    ; if (x < -3) jump to label1
cmp    rdi, 2
jg     .label1    ; if (x > 2) jump to label1
xor     rax, rax   ; if (-3 <= x <= 2) rax = 0
jmp     .label2

.label1:
mov     eax, 1     ; if (x < -3 || x > 2) rax = 1
.label2:
...
```

x86 Stack Operations: PUSH and POP

Reminder: the Stack grows downwards!

PUSH imm/r/m

- 1 Decrements stack pointer (ESP/RSP)
 - 4 bytes in 32 bit mode, 8 bytes in 64 bit
- 2 Writes operand to top of stack

POP r/m

- 1 Reads value from top of stack
- 2 Stores value into operand (register or memory)
- 3 Increments stack pointer (ESP/RSP)
 - 4 bytes in 32 bit mode, 8 bytes in 64 bit

Quiz 6: PUSH and POP

```
push 0x11
push 0x22
push 0x33
pop rcx
pop rbx
pop rax
push rax
cmp rax, [rsp]
je .label1
xor rax, rax
.label1:
...
```

Question: What is the final value of `rax`?

- ① 0x11
- ② 0x22
- ③ 0x33
- ④ 0x00

Quiz 6: solution

```
push 0x11
push 0x22
push 0x33
pop rcx      ; rcx = 0x33, stack top now points to 0x22
pop rbx      ; rbx = 0x22, stack top now points to 0x11
pop rax      ; rax = 0x11, stack is now empty
push rax     ; push 0x11 back to stack, stack top now points to 0x11
cmp rax, [rsp] ; compare rax (0x11) with value at stack top (0x11)
je .label1   ; jump to label1 since they are equal
xor rax, rax  ; not executed
.label1:
...
```

Correct answer: 1. 0x11

x86 Control Flow: CALL and RET

CALL imm/r/m

- ① PUSH EIP|RIP
 - Reminder: the IP points to the address to the *next* instruction
- ② JMP imm/r/m $\implies$ Transfers control to target operand
 - Direct: via immediate or EIP|RIP-relative
 - relative: distance (positive or negative) from the address of the IP
 - Indirect: via register or memory

Direct call destinations are statically known.

Indirect ones are determined dynamically at runtime $\implies$ harder to analyze and predict

RET imm?

- POP EIP|RIP $\implies$ Transfers control back to caller
- Optional **imm?** operand: RET *n*
 - not only pops the return address from the stack
 - also adds *n* to the stack pointer afterwards

Quiz 7: CALL and RET

```
xor rax, rax  
call foo  
add rax, 1  
jmp done
```

```
foo:  
    add rax, 5  
    ret
```

```
done:  
    ...
```

Question: What is the final value of `rax`?

Quiz 7: solution

```
xor rax, rax    ; rax = 0
call foo        ; push address of next instruction to stack and jumps to foo
add rax, 1      ; rax = rax + 1 = 5 + 1 = 6
jmp done

foo:
add rax, 5      ; rax = rax + 5 = 0 + 5 = 5
ret             ; pop return address to rip (ip points to instruction after call)

done:
...
```

Correct answer: 0x6

x86 (some) Control Flow Tricks

Get Instruction Pointer (EIP/RIP)

- x86-32 cannot directly read the instruction pointer
 - in x86-64 you can use: `lea rax, [rip+0]`
- Trick: CALL pushes the return address (next instruction)
- Then POP retrieves it into a register

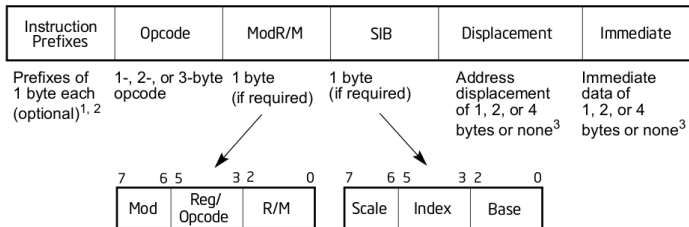
```
call $+5      ; call "next instruction" (5 = size of CALL)
pop eax       ; eax = address of this "pop eax"
```

PUSH + RET as a Jump

- RET pops the destination address from the stack
- Combine with PUSH to perform an indirect jump

```
push target_addr
ret
```

Instruction format: from 1 byte to 15 bytes



- **Prefixes** (optional): change behavior or size. In 64-bit code, *REX* (Register EXtension) can enable 64-bit operands and extra registers
- **Opcode** (always): the actual operation
- **ModR/M** (often): selects (which) register vs memory
- **SIB** (only if needed): scale/index/base for complex addressing
- **Displacement** (optional): address offset; typically 8 or 32 bits
- **Immediate** (optional): constant encoded with the instruction

Rule of thumb: If there's no memory operand, SIB/Displacement are absent.

E.g., MOV Byte Encoding Walkthrough in x86-64

`mov rax, [r12 + rbx*4 + 0x12345678]` → 49 8B 84 9C 78 56 34 12

Byte(s)	Field	Meaning
49	REX	"64bit instr. – uses the upper bank r8–r15"
8B	Opcode	MOV r64, r/m64
84	ModR/M	mod=(disp32 present), reg=RAX , SIB follows
9C	SIB	scale= $\times 4$, index=RBX, base=R12
78 56 34 12	disp32	Little-endian of 0x12345678

x86 NOPs: Single- and Multi-Byte

No-Operation instructions

- The 90h is the canonical **1-byte** NOP
 - Historically encodes XCHG RAX, RAX
 - Modern CPUs treat it as a true no-op
- x86 provides **multi-byte** NOPs (2-9 bytes)
 - *Look* like memory ops but the CPU does not access memory
- Used for: **alignment**, **padding** or **patches**

Len	Bytes (hex)	Typical disassembly
1	90	<code>nop</code>
2	66 90	<code>nop</code>
3	0F 1F 00	<code>nop dword ptr [rax]</code>
4	0F 1F 40 00	<code>nop dword ptr [rax+0]</code>
5	0F 1F 44 00 00	<code>nop dword ptr [rax+rax+0]</code>
6	66 0F 1F 44 00 00	<code>nop word ptr [rax+rax+0]</code>
7	0F 1F 80 00 00 00 00	<code>nop dword ptr [rax+00000000]</code>
8	0F 1F 84 00 00 00 00 00	<code>nop dword ptr [rax+rax+00000000]</code>
9	66 0F 1F 84 00 00 00 00 00	<code>nop word ptr [rax+rax+00000000]</code>

This is how a “real” computer program (x86-64) looks 🤪

```
0F 00 48 89 44 24 20 48 8D 05 6A AD 0F 00 48 89
44 24 28 48 8D 05 76 AD 0F 00 48 89 44 24 30 48
8D 05 82 AD 0F 00 48 89 44 24 38 48 8D 05 8E AD
0F 00 48 89 44 24 40 48 8D 05 9A AD 0F 00 49 89
43 80 48 8D 05 A7 AD 0F 00 49 89 43 88 48 8D 05
B4 AD 0F 00 49 89 43 90 48 8D 05 C1 AD 0F 00 49
89 43 98 48 8D 05 CE AD 0F 00 49 89 43 A0 48 8D
05 DB AD 0F 00 49 89 43 A8 48 8D 05 E8 AD 0F 00
49 89 43 B0 48 8D 05 F5 AD 0F 00 49 89 43 B8 48
8D 05 0A AE 0F 00 49 89 43 C0 48 8D 05 1F AE 0F
00 49 89 43 C8 48 8D 05 34 AE 0F 00 49 89 43 D0
48 8D 05 49 AE 0F 00 49 89 43 D8 48 8D 05 5E AE
0F 00 49 89 43 E0 48 8D 05 73 AE 0F 00 49 89 43
E8 48 8D 05 80 AE 0F 00 49 89 43 F0 0F AE E8 B9
A0 00 00 00 E8 B7 82 0B 00 41 B8 A0 00 00 00 48
89 05 E2 48 15 00 48 8D 54 24 20 48 89 05 DE 48
15 00 48 8B C8 48 8D 98 A0 00 00 00 48 89 1D D5
```

Quiz 8: CALL, RET and size of the instructions

```
xor rax, rax
call foo
add rax, 1 ; Encoded as: 48 83 C0 01
jmp done
```

foo:

```
add qword ptr [rsp], 4
add rax, 5
ret
```

done:

...

Question: What is the final value of `rax`?

Quiz 8: solution

```
xor rax, rax    ; rax = 0
call foo        ; push address of next instruction to stack and jumps to foo
add rax, 1      ; not executed
jmp done

foo:
add qword ptr [rsp], 4    ; add 4 to the return address
                        ; skips the next 4 bytes instruction (add rax, 1)
add rax, 5        ; rax = rax + 5 = 0 + 5 = 5
ret              ; pop return address to rip (ip points to instruction after call)

done:
...
```

Correct answer: 0x5

Shellcode

Piece of **machine code** used as **payload**

- Injected via a memory corruption vulnerability (e.g., buffer overflow) to exploit a software vulnerability
- Executes **self-contained** (without external dependencies) **position-independent** (independent of its load address) code
- Often performs two non-exclusive actions:
 - spawn a (remote) shell
 - download and execute code
- Often constrained:
 - limited size (**small**)
 - restricted byte values (e.g., no NULL bytes).

Shellcode term is used loosely

Used for small, in-memory, position-independent code used as a loader or execution stage, even outside of the context of a classic vulnerability exploit

Many ASCII characters correspond to valid x86 instructions

- Train an **n-gram language model** on large English corpora
- Search for word sequences that
 - look like English
 - considered together are valid machine instructions
- Self-modifying decoder which reconstructs the *real* payload at runtime

Automatically transform arbitrary x86_32 shellcode into **executable English-looking prose** 🤖

English Shellcode [MSMM09]

	ASSEMBLY	OPCODE	ASCII
1	push %esp push \$20657265 imul %esi,20(%ebx),\$616D2061 push \$6F jb short \$22	54 68 65726520 6973 20 61206D61 6A 6F 72 20	There is a major
2	push \$20736120 push %ebx je short \$63 jb short \$22	68 20617320 53 74 61 72 20	h as Star
3	push %ebx push \$202E776F push %esp push \$6F662065 jb short \$6F	53 68 6F772E20 54 68 6520666F 72 6D	Show. The form
4	push %ebx je short \$63 je short \$67 jnb short \$22 inc %esp jb short \$77	53 74 61 74 65 73 20 44 72 75	States Dru
5	popad	61	a

1	Skip	2	Skip
There is a major	center of economic activity, such	as Star	Trek, including The Ed
Skip	3	Skip	
Sullivan	Show. The former	Soviet Union. International organization participation	
Skip		4	Skip
Asian Development Bank, established in the United	States Drug	Enforcement	

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

Application Binary Interface (ABI)

Low-level “contract” between compiled programs and the OS

- Specifies how binaries interact at runtime
 - **Calling conventions:** which registers/stack locations hold arguments and return values
 - **Register usage:** caller-saved vs callee-saved registers
 - **Data types & alignment:** size, layout, and padding rules in memory
 - **System call interface:** how user programs invoke OS services
 - **Binary format:** e.g., ELF (Linux), PE (Windows), Mach-O (macOS)
- Importance:
 - Ensures interoperability of code w.r.t. compilers/languages
 - Enables linking with system libraries without source code

Process $\longleftrightarrow$ OS

Calling conventions, registers, and syscalls **MUST** be respected with the OS.

Within a single program, the compiler is free to choose its own internal conventions.

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

x86_32 Calling Conventions

The most frequent are:

- **cdecl**, the C convention, frequent in Linux/GCC
 - the caller is responsible to clean up the stack
 - supports variable number of arguments
 - E.g., `int printf(const char *restrict format, ...)`
- **stdcall**, the “standard” convention, frequent with Windows API
 - the callee is responsible to clean up the stack
 - $\implies$ less code, callers do not need to do anything,
 - but supports only fixed number of arguments

All the rest is in common

- Pass the arguments in reverse order (right to left) on the stack
- Register saving
- Return values convention
 - Integer/pointer ≤ 32 bits: **EAX**
 - 64-bit integer: split across **EDX:EAX** (**EDX** = high 32b, **EAX** = low 32b)

Caller-saved vs. callee-saved registers

- Caller-saved (volatile)
 - EAX, ECX, EDX
 - The caller must assume they may be changed across a call
 - If the caller needs their values preserved, it must save/restore them
- Callee-saved (non-volatile)
 - EBX, ESI, EDI, EBP
 - The callee must restore these before returning

Of course, ESP is always maintained...

Example – sum.c

```
#include <stdio.h>

int dumbsum4args(int w, int x, int y, int z) {
    // Caller: dumbsum8args
    // Callee: dumbsum4args
    return w+x+y+z;
}

int dumbsum8args(int a, int b, int c, int d, int e, int f, int g, int h) {
    // Caller: main
    // Callee: dumbsum8args
    int partial_sum = dumbsum4args(a,b,c,d);
    partial_sum += dumbsum4args(e,f,g,h);
    return partial_sum;
}

int main() {
    printf("Sum=%d\n", dumbsum8args(1, 2, 4, 8, 16, 32, 64, 128));
}
```

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

System V AMD64 ABI

Used in Unix-like OS: Linux, macOS, BSD, etc.

- First 6 args: **RDI**, **RSI**, **RDX**, **RCX**, **R8**, **R9**
- Extra args on stack
- Return in **RAX**
- Caller cleans up stack

```
push    128                ; h
push    64                 ; g
mov     r9d, 32            ; f
mov     r8d, 16            ; e
mov     ecx, 8             ; d
mov     edx, 4             ; c
mov     esi, 2             ; b
mov     edi, 1             ; a
call    dumbsum8args
add     rsp, 10h           ; Caller cleans up stack
mov     esi, eax
mov     edi, offset format ; "Sum=%d\n"
mov     eax, 0
```

Microsoft x64 ABI (Windows)

- First 4 args: `RCX`, `RDX`, `R8`, `R9`
- Extra args on stack
- Return in `RAX`
- Callee cleans up stack

```
sub    rsp, 48h
mov    dword ptr [rsp+38h], 128
mov    dword ptr [rsp+30h], 64
mov    dword ptr [rsp+28h], 32
mov    dword ptr [rsp+20h], 16
mov    r9d, 8
mov    r8d, 4
mov    edx, 2
mov    ecx, 1
call   dumbsum8args
mov    edx, eax
lea    rcx, Format      ; "Sum=%d\n"
```

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

System calls

Normal programmers are not concerned by *syscalls*; for example:

- `#include <stdio.h>`
 - Tells the preprocessor to look in the system directories
 - Where the standard library headers ($\implies$ just signature not code) live
- `fp = fopen("example.txt", "r");`
 - `fopen` is implemented (code) in the C standard library
 - Windows: `ucrtbase.dll`
 - macOS: `libSystem.B.dylib`
 - Linux/Android: `libc.so` but different implementations (glibc/bionic)
 - The linker bundles the libraries into the final executable
 - Static $\rightarrow$ all the code in the same (big) executable
 - Dynamic $\rightarrow$ load the needed libraries at runtime
- The *libcall* to `fopen` follows the ABI
- Finally, the library makes the *syscall*
 - A different convention applies for each operating system/architecture

Syscall Differences: Linux vs. Windows (32/64-bit)

Aspect	Linux		Windows	
	32-bit	64-bit	32-bit	64-bit
x86 Instruction	int 0x80/sysenter	syscall	sysenter	syscall
Syscall number register	EAX	RAX	EAX	R10
Argument passing	EBX, ECX, EDX, ESI, EDI, EBP	RDI, RSI, RDX, R10, R8, R9	Stack	RCX, RDX, R8, R9, stack
Return value	EAX	RAX	EAX	RAX

Syscalls tables: <https://syscalls.w3challs.com/>

In Linux syscall numbers are stable, in Windows not at all!

Linux: numbers different in 32/64 bits, but stable across kernel versions

Windows: numbers are undocumented and version-dependent

System call wrappers (Linux & Windows, 32/64-bit)

- A wrapper function *w* uses a tiny assembly stub to:
 - place the **syscall number** in the designated register
 - marshal **arguments** per the OS/ABI
 - transfer control to kernel (**trap**)
- In kernel mode, the syscall handler:
 - validates the syscall number and arguments
 - calls the appropriate kernel routine
 - places the **result** in a return register (EAX/RAX)
 - returns to user mode with the matching fast-path return sequence
- Back in user mode, *w* interprets the result:
 - Linux: on error, kernel returns `-errno` in RAX/EAX; the C wrapper sets `errno` and returns `-1`
 - Windows: kernel returns an `NTSTATUS` in RAX/EAX; Win32 API stubs (e.g., `kernel32`) map this to `GetLastError()`

Performance note

System calls are typically one/two orders of magnitude *slower* than a user-space function call, due to the user↔kernel transition and validation

Syscall example (32 bit, Intel syntax) under Linux

```
section .text
global _start

_start:

    mov eax, 4      ; use the write syscall
    mov ebx, 1      ; write to stdout
    mov ecx, msg    ; use string "Hello World"
    mov edx, msglen ; write 12 characters
    int 0x80        ; make syscall

    mov eax, 1      ; use the exit syscall
    mov ebx, 0      ; status code 0
    int 0x80        ; make syscall

section .data

msg:    db "Hello World", 10
msglen equ $-msg
```

To assemble and link:

```
nasm -f elf32 ./hello32-intel.asm && ld -m elf_i386 -o hello32-intel hello32-intel.o
```

Outline

- 1 Introduction
- 2 Architecture and Instructions
- 3 Application Binary Interface (ABI)
 - x86_32
 - x86_64
- 4 System Calls
 - User-Mode Hooking and Syscall-Based Evasion in Windows

Problem: EDR¹ / AV² Hooking

- In Windows, security products often hook **user-mode APIs**
- Common techniques:
 - IAT patching
 - DLL injection
 - Inline hooking
 - E.g., overwriting the prologue of a function with a jump to a hook
- Hooks commonly placed in `ntdll.dll`
- Goal:
 - Intercept sensitive calls (process injection, file writes, network connections, etc.)
 - $\implies$ attackers using standard APIs are easily monitored in user mode

Why `ntdll.dll`?

`ntdll.dll` is the user-mode façade of the Windows kernel: it exposes the native API, performs syscall transitions, and underpins core runtime functionality in every process.

¹Endpoint Detection and Response

²AntiVirus

E.g., NtCreateFile syscall wrapper in ntdll.dll 64bit

```
MOV    R10, RCX    ; First arg in RCX, but syscall expects it in R10
MOV    EAX, 0x55    ; System Service Number (SSN) for NtCreateFile
TEST   byte ptr [7ffe0308], 0x1 ; Check if alternative stub should be used
JNZ    0xabd0fe7    ; Go to alternative syscall stub
SYSCALL                ; invoke syscall 0x55 (NtCreateFile on that build)
RET
```

Let's insert a hook to monitor which files this process creates...

so we modify (in memory, not on disk) this code by adding a callback →

E.g., Adding a hook in NtCreateFile

- We cannot use as many instructions as we want
- Must stay within the available byte limit (8 bytes total for the two MOVs)
- Using a CALL (5 bytes) + 3 NOPs (padding)

```
CALL hook          ; near x64 call is 5 bytes
NOP                ; 1 byte padding
NOP                ; 1 byte padding
NOP                ; 1 byte padding
TEST byte ptr [7ffe0308], 0x1
JNZ 0xabd0fe7
SYSCALL            ; invoke syscall 0x55
RET
```

hook:

```
    PUSH RCX        ; save RCX
    ; custom logic, e.g., inspect syscall arguments
    POP  RCX        ; restore RCX, and the other registers
    MOV  R10, RCX    ; 3 bytes
    MOV  EAX, 0x55    ; 5 bytes
    RET
```

How can you get around the hooks? (1/2)

- Remove them by restoring the original instructions
 - Pros:
 - Simple and conceptually straightforward
 - Cons:
 - Requires knowing the exact original bytes (version-dependent)
 - Hooks may be reinstalled dynamically by EDR
 - Modifying `ntdll` in memory is suspicious

How can you get around the hooks? (2/2)

- Invoke the `syscall` **directly**, i.e., from user code
 - **Pros:**
 - No reliance on `ntdll` integrity
 - Fast and minimal execution path
 - **Cons:**
 - Syscall numbers are Windows-version dependent
 - May be detected at kernel level (e.g., via return address or call stack inspection)
 - E.g. code: <https://github.com/klezVirus/SysWhispers3>
- Invoke the `syscall` **indirectly**, i.e., bypassing hooked prologue
 - **Pros:**
 - Avoids executing hooked `ntdll` stubs
 - More stealthy (reuses legitimate `syscall` instructions)
 - Harder to detect via simple user-mode heuristics
 - **Cons:**
 - More complex (stub discovery, address resolution)
 - Kernel-level monitoring can still detect anomalies (e.g., stack frames)
 - E.g. code: <https://github.com/thefLink/RecycledGate>

- Linux system call list:
<https://syscalls.w3challs.com/>
- Linux User/Kernel ABI: the realities of how C and C++ programs really talk to the OS
by Greg Law @ ACCU 2018
<https://www.youtube.com/watch?v=4CdmGxc5BpU>
- “The C++ ABI From the Ground Up”
by Louis Dionne @ CppCon2019
<https://youtu.be/DZ931P1I7wU>

- [B⁺06] Daniel Bilar et al.
Statistical structures: Fingerprinting malware for classification and analysis.
Proceedings of Black Hat Federal 2006, 2006.
- [Dol13] Stephen Dolan.
mov is turing-complete.
Technical report, University of Cambridge, 2013.
Available at <https://github.com/LeetArxiv/ImplementedPapers/blob/main/mov-is-turing-complete.pdf>.
- [MSMM09] Joshua Mason, Sam Small, Fabian Monroe, and Greg MacManus.
English shellcode.
In *Proceedings of the 16th ACM conference on Computer and communications security*, pages 524–533, 2009.